Representación del espacio de búsqueda y Optimización por Colonia de Hormigas (ACO)

aplicadas al problema del ciclo hamiltoniano

Osvaldo Fabian Ramiro Balboa

Noviembre de 2026

Resumen

Este documento reúne la investigación previa (fase 1 del aula invertida) sobre dos temas: cómo representar el grafo y las rutas candidatas sin producir soluciones ilegales, y cómo funciona la Optimización por Colonia de Hormigas (ACO). Se incluye un ejemplo didáctico de siete vértices, el pseudocódigo del método adaptado al ciclo hamiltoniano, las decisiones de diseño posibles con sus dificultades y las fuentes que respaldan cada afirmación. Los números del ejemplo son ilustrativos y *no* son resultados experimentales sobre el FHCP.

Cómo leer este documento. Para separar lo que pide la práctica de lo que proponemos como equipo, algunas frases llevan una etiqueta: **[Requisito]** lo exige el enunciado, **[Propuesta]** es una opción de diseño sugerida por las fuentes y **[Duda]** es una pregunta abierta para resolver con el profesor o con el equipo. Las referencias entre corchetes, como [1], remiten a la sección 7.

Índice

1. Términos básicos **2**

2. Tema general: representación del espacio de búsqueda **2**
- 2.1. Cómo guardar el grafo . . . 2
- 2.2. Cómo representar una ruta . . . 3
- 2.3. Por qué una permutación válida puede no ser un ciclo . . . 3
- 2.4. Cómo tratar rutas que no cumplen las reglas . . . 4

3. Algoritmo: Optimización por Colonia de Hormigas (ACO) **4**
- 3.1. Idea . . . 4
- 3.2. Construcción de rutas sin repetir vértices . . . 4
- 3.3. Feromonas: refuerzo y evaporación . . . 5
- 3.4. Información del grafo para elegir el siguiente paso . . . 5
- 3.5. Callejones sin salida y ciclos que no cierran . . . 5

4. Ejemplo didáctico con siete vértices **6**

5. Pseudocódigo y ajustes principales **8**

6. Aplicación al ciclo hamiltoniano: opciones, dificultades y dudas **9**
6.1. Opciones de diseño 9
6.2. Dificultades previstas 9
6.3. Dudas pendientes 10

7. Fuentes consultadas **10**

1. Términos básicos

Definición 1 (Grafo)**.** Un grafo no dirigido es un par $G = (V, E)$, donde V es un conjunto de n *vértices* y E es un conjunto de m *aristas*; cada arista $\{u, v\}$ une dos vértices distintos. Dos vértices unidos por una arista son *vecinos*; el *grado* de un vértice es su número de vecinos [6].

Definición 2 (Ciclo hamiltoniano)**.** Un ciclo hamiltoniano es una sucesión $(v_1, v_2, \ldots, v_n)$ que contiene cada vértice de V exactamente una vez, tal que $\{v_i, v_{i+1}\} \in E$ para $i = 1, \ldots, n-1$ y, además, $\{v_n, v_1\} \in E$ (el *cierre*). Tiene exactamente n aristas. Decidir si un grafo tiene uno es un problema NP-completo [7], por eso se usan metaheurísticas.

Definición 3 (Permutación)**.** Una permutación de V es un ordenamiento de sus n vértices en el que ninguno se repite ni falta. Existen $n!$ permutaciones; si se fija el vértice inicial y se ignora el sentido del recorrido, quedan $(n-1)!/2$ ciclos distintos posibles.

Definición 4 (Espacio de búsqueda y factibilidad)**.** El *espacio de búsqueda* es el conjunto de todas las soluciones candidatas que el algoritmo puede generar con la representación elegida. Una candidata es *factible* si cumple todas las reglas del problema; en este caso, si es un ciclo hamiltoniano.

2. Tema general: representación del espacio de búsqueda

La representación decide qué estructuras puede producir el algoritmo y, por lo tanto, qué tan fácil es caer en soluciones ilegales. Larrañaga et al. [4] muestran, para el problema del viajero (TSP), que la misma metaheurística cambia mucho de comportamiento según la codificación y los operadores. Hay que representar dos cosas distintas: el *grafo* (los datos del problema) y la *ruta* (la solución candidata).

2.1. Cómo guardar el grafo

Matriz de adyacencia. Es una tabla A de $n \times n$ con $A[u][v] = 1$ si la arista $\{u, v\}$ existe y 0 si no. Consultar si dos vértices están conectados cuesta $O(1)$, pero la memoria es $O(n^2)$ aunque el grafo tenga pocas aristas [6].

Lista de adyacencia. Para cada vértice se guarda la lista de sus vecinos. Usa memoria $O(n+m)$ y permite recorrer los vecinos de un vértice en tiempo proporcional a su grado, que es justo lo que hace una hormiga al elegir el siguiente paso. Saber si existe una arista concreta cuesta $O(\text{grado})$ [6].

Dimensión real del FHCP. Las 1001 instancias del FHCP tienen entre 66 y 9528 vértices [8, 9]. Para $n = 9528$, una matriz tiene $9528^2 \approx 9.08 \times 10^7$ celdas: unos 91 MB si se usa un byte por celda (unos 11 MB si se usa un bit). Es manejable, pero desperdicia memoria si el grafo es disperso.

[Propuesta] Usar **listas de adyacencia** para recorrer vecinos y, adicionalmente, un **conjunto de aristas** (tabla hash de pares ordenados (mín, máx)) para verificar en $O(1)$ el cierre y la validez de una ruta. **[Duda]** Medir el grado medio de las instancias al descargarlas para confirmar que son dispersas.

Tabla 1: Comparación de representaciones del grafo.

	Matriz de adyacencia	Lista de adyacencia
Memoria	$O(n^2)$	$O(n+m)$
¿Existe $\{u, v\}$?	$O(1)$	$O(\text{grado}(u))$
Recorrer vecinos de u	$O(n)$	$O(\text{grado}(u))$
Adecuada para	grafos densos o pequeños	grafos dispersos

Formato de los archivos. El FHCP usa el formato HCP [9]: una cabecera, la lista de aristas (una por línea) y un cierre. Las soluciones usan el mismo estilo con `TYPE : TOUR` y una `TOUR_SECTION` con los vértices en el orden del ciclo, también terminada en `-1`. Esquema (los valores son ilustrativos):

```
NAME : graph1
COMMENT : ...
TYPE : HCP
DIMENSION : 66
EDGE_DATA_FORMAT : EDGE_LIST
EDGE_DATA_SECTION
1 2
...
-1
EOF
```

[Duda] Confirmar con un archivo real si los vértices se numeran desde 1.

2.2. Cómo representar una ruta

[Requisito] La práctica pide estructuras que representen permutaciones de los vértices sin generar «cromosomas» ilegales, y que cada programa devuelva un arreglo de vértices como solución propuesta.

[Propuesta] La representación más directa es la **permutación de vértices** (*path representation* en [4]): un arreglo de longitud n como $[1, 7, 6, 5, 4, 3, 2]$, que se lee como el recorrido $1 \to 7 \to 6 \to 5 \to 4 \to 3 \to 2 \to 1$. Garantiza por construcción que cada vértice aparece una sola vez.

Otras codificaciones descritas en [4] y útiles para el equipo:

- **Adyacencia**: la posición i guarda el sucesor del vértice i.
- **Ordinal**: cada gen indica una posición en una lista de vértices restantes; cualquier cruza de un punto produce una ruta válida.
- **Llaves aleatorias** (*random keys*) [11]: un vector de números reales que se ordena para obtener la permutación. Es la puerta de entrada natural para algoritmos continuos como DE y PSO (integrantes 4 y 5).

2.3. Por qué una permutación válida puede no ser un ciclo

Una permutación cumple dos de las cuatro reglas del ciclo hamiltoniano: *están todos los vértices* y *ninguno se repite*. Pero no garantiza las otras dos:

1. que cada par de vértices consecutivos esté unido por una arista del grafo, y
2. que exista la arista de *cierre* entre el último y el primero.

En el TSP clásico el grafo es completo y toda permutación es una ruta; en el ciclo hamiltoniano el grafo es incompleto (y en el FHCP, deliberadamente engañoso [8]), así que la mayoría de las permutaciones usa aristas inexistentes.

2.4. Cómo tratar rutas que no cumplen las reglas

Coello Coello [5] clasifica las técnicas de manejo de restricciones; las tres que aplican aquí:

1. **Evitar** (representación y operadores que preservan factibilidad): la ruta solo avanza por aristas existentes. ACO lo hace de forma natural, porque cada hormiga elige entre los *vecinos* del vértice actual.

2. **Penalizar**: se aceptan permutaciones arbitrarias y se mide cuánto les falta. **[Propuesta]** Función de costo
$$f(\pi) = \sum_{i=1}^{n} \big[\{\pi_i, \pi_{(i \bmod n)+1}\} \notin E\big],$$
es decir, el número de aristas faltantes incluido el cierre. Vale 0 si y solo si π es un ciclo hamiltoniano, y sirve como objetivo común para comparar GA, SA, DE y PSO.

3. **Reparar**: transformar una candidata ilegal en una mejor. Por ejemplo, la *rotación de Pósa* [10] toma un camino $v_1 \dots v_k$ cuyo extremo v_k es vecino de algún v_i, y lo reescribe como $v_1 \dots v_i\, v_k v_{k-1} \dots v_{i+1}$, que tiene un extremo nuevo desde el cual intentar extender o cerrar.

Comprobación de una ruta (la misma para todos los algoritmos). Dada $\pi = (\pi_1, \dots, \pi_n)$: (a) su longitud es n; (b) no hay repetidos (un conjunto de tamaño n); (c) cada par consecutivo es arista; (d) $\{\pi_n, \pi_1\}$ es arista. Cuesta $O(n)$ con el conjunto de aristas.

3. Algoritmo: Optimización por Colonia de Hormigas (ACO)

3.1. Idea

Las hormigas reales depositan feromona al caminar; los caminos cortos se recorren más veces por unidad de tiempo, acumulan más feromona y atraen a más hormigas. ACO imita este mecanismo: un conjunto de agentes construye soluciones paso a paso y se comunica de forma indirecta (*estigmergia*) a través de valores numéricos asociados a las aristas [1, 3]. La primera versión fue el *Ant System* (AS) [1]; luego aparecieron variantes como *Ant Colony System* (ACS) [2] y *MAX–MIN Ant System* (MMAS) [3, 13].

3.2. Construcción de rutas sin repetir vértices

Cada hormiga k empieza en un vértice y mantiene una *memoria* (lista tabú) de los vértices ya visitados. Estando en i, sus opciones son los vecinos aún no visitados N_i^k. En AS elige $j \in N_i^k$ con probabilidad [1, 3]
$$p_{ij}^k = \frac{\tau_{ij}^{\alpha}\, \eta_{ij}^{\beta}}{\sum_{l \in N_i^k} \tau_{il}^{\alpha}\, \eta_{il}^{\beta}},$$
donde τ_{ij} es la feromona de la arista, η_{ij} es la información heurística, y α, β ponderan una contra otra. Como solo se consideran vecinos no visitados, **la hormiga nunca repite vértices ni usa aristas inexistentes**: la representación evita por construcción las soluciones ilegales.

3.3. Feromonas: refuerzo y evaporación

La feromona τ_{ij} es la memoria colectiva: cuánto ha convenido usar la arista $\{i, j\}$ en el pasado. Tras cada iteración [1, 3]:

$$\tau_{ij} \leftarrow (1 - \rho)\, \tau_{ij} + \sum_{k} \Delta\tau_{ij}^{k}.$$

- **Evaporación** ($0 < \rho \leq 1$): todas las aristas pierden una fracción de su feromona. Evita que la colonia se quede para siempre con las primeras rutas encontradas.
- **Depósito** ($\Delta\tau_{ij}^{k}$): las hormigas refuerzan las aristas que usaron, en proporción a la calidad de su ruta. En el TSP, $\Delta\tau = Q/L_k$ (L_k = longitud del recorrido).

Variantes: en ACS solo la mejor ruta deposita y además hay una actualización local que reduce la feromona de las aristas recién usadas para diversificar [2]; en MMAS solo deposita la mejor hormiga y τ se limita a $[\tau_{\text{mín}}, \tau_{\text{máx}}]$ para evitar el estancamiento [13].

3.4. Información del grafo para elegir el siguiente paso

En el TSP la heurística es $\eta_{ij} = 1/d_{ij}$ (preferir ciudades cercanas). En el ciclo hamiltoniano no hay distancias, así que hay que usar la *estructura* del grafo. **[Propuesta]** Una opción natural es la regla de Warnsdorff (preferir el vecino con menos vecinos libres):

$$\eta_{ij} = \frac{1}{\max\bigl(1,\ |N(j) \setminus \text{visitados}|\bigr)}.$$

La intuición: un vértice con pocos vecinos libres corre peligro de quedar aislado, así que conviene visitarlo pronto. Otras señales útiles: si un vértice tiene grado 2, sus dos aristas son *obligatorias* en cualquier ciclo hamiltoniano, y conviene evitar los movimientos que dejen algún vértice libre sin vecinos libres. **[Duda]** La regla de Warnsdorff proviene del recorrido del caballo; no encontramos todavía una fuente que la evalúe con ACO sobre el FHCP.

3.5. Callejones sin salida y ciclos que no cierran

Al trabajar sobre un grafo incompleto aparecen dos fallas que no existen en el TSP:

1. **Callejón sin salida**: la hormiga llega a un vértice cuyos vecinos ya fueron visitados, pero todavía faltan vértices. Su ruta es un camino de $k < n$ vértices.

2. **Camino hamiltoniano sin cierre**: visita los n vértices, pero el último no es vecino del primero.

Opciones para tratarlas:

- **[Propuesta]** *Puntuar* la ruta parcial con el número de aristas válidas recorridas (incluido el cierre si existe) dividido entre n, y depositar feromona en proporción a ese valor. Así una hormiga casi exitosa aporta información útil.
- **[Propuesta]** *Reparar* con rotaciones de Pósa [10] antes de evaluar: si el extremo tiene un vecino ya visitado, rotar el camino y seguir extendiendo o intentar cerrar.
- **[Propuesta]** *Retroceso limitado* (*backtracking*): deshacer los últimos pasos y probar otro vecino, con un tope de intentos para no convertirlo en fuerza bruta.
- Wagner y Bruckstein [12] propusieron *Hamiltonian(t)*, un proceso de una sola hormiga que marca vértices y elige según las marcas vecinas; muestran que los ciclos hamiltonianos son ciclos límite de ese proceso. Es un antecedente directo de hormigas aplicadas a este problema.

4. Ejemplo didáctico con siete vértices

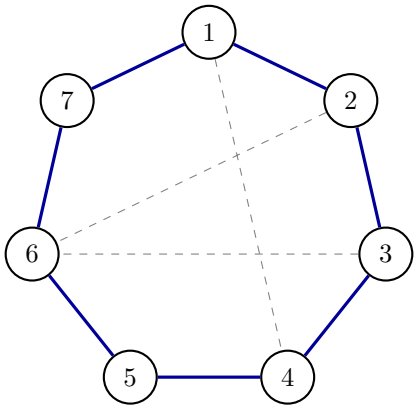


Figura 1: Grafo de ejemplo con $n = 7$ y $m = 10$. En azul, el único ciclo hamiltoniano; en gris discontinuo, las tres aristas «trampa» $\{1,4\}$, $\{2,6\}$ y $\{3,6\}$.

Representación. Lista de adyacencia: $1:\{2,4,7\}$, $2:\{1,3,6\}$, $3:\{2,4,6\}$, $4:\{1,3,5\}$, $5:\{4,6\}$, $6:\{2,3,5,7\}$, $7:\{1,6\}$.

Razonamiento estructural. Los vértices 5 y 7 tienen grado 2, así que las aristas $\{4,5\}$, $\{5,6\}$, $\{6,7\}$ y $\{7,1\}$ son obligatorias. Entonces el vértice 6 ya usa dos aristas (hacia 5 y 7) y quedan descartadas $\{2,6\}$ y $\{3,6\}$. Falta unir 1 con 4 pasando por 2 y 3: solo es posible como $1{-}2{-}3{-}4$. El ciclo hamiltoniano es único: 1–2–3–4–5–6–7–1 (lo confirmamos enumerando las 6! permutaciones que empiezan en 1: solo esta y su recorrido inverso tienen costo 0). Este es el mismo tipo de estructura que vuelve difíciles a las instancias del FHCP: pocas aristas correctas escondidas entre muchas tentadoras [8].

Costo de permutaciones (función de penalización).

Permutación	Aristas faltantes	Motivo
$[1,4,3,6,7,2,5]$	3	faltan $\{7,2\}$, $\{2,5\}$ y el cierre $\{5,1\}$
$[2,3,4,1,7,6,5]$	1	todas existen salvo el cierre $\{5,2\}$
$[1,2,3,4,5,6,7]$	0	ciclo hamiltoniano

Primer paso de una hormiga. Parámetros $\alpha = 1$, $\beta = 2$, $\tau_0 = 1$ en todas las aristas y la heurística de Warnsdorff. Desde el vértice 1, los vecinos libres de 2, 4 y 7 son 2, 2 y 1, respectivamente, así que $\eta = (\frac{1}{2}, \frac{1}{2}, 1)$ y

$$p_{12} = \frac{0.25}{0.25 + 0.25 + 1} = 0.167, \quad p_{14} = 0.167, \quad p_{17} = 0.667.$$

La heurística empuja hacia el vértice 7, de grado 2, que es correcto. Desde 7 el único vecino libre es 6. En 6 (visitados $\{1,7,6\}$) los candidatos son 2, 3 y 5 con $\eta = (1, \frac{1}{2}, 1)$, de modo que $p_{62} = 0.444$, $p_{63} = 0.111$, $p_{65} = 0.444$.

Tres hormigas de la primera iteración.

- **Hormiga A**: 1–7–6–2–3–4–5. Visita los 7 vértices, pero $\{5, 1\}$ no existe: *camino hamiltoniano sin cierre*. 6 aristas válidas.
- **Hormiga B**: $1-4-3-6-7$. En 7 sus vecinos (1 y 6) ya fueron visitados y faltan 2 y 5: *callejón sin salida*. 4 aristas válidas.
- **Hormiga C**: $1-7-6-5-4-3-2-1$. Ciclo hamiltoniano: 7 aristas válidas y cierre correcto.

Comprobación de C: longitud 7, sin repetidos, las aristas $\{1, 7\}, \{7, 6\}, \{6, 5\}, \{5, 4\}, \{4, 3\}, \{3, 2\}$ existen y el cierre $\{2, 1\}$ también.

Actualización de feromonas con $\rho = 0.5$ y depósito $\Delta\tau = (\text{aristas válidas})/n$: A deposita $6/7 = 0.857$, B $4/7 = 0.571$ y C 1.

Arista	1-2	1-4	1-7	2-3	2-6	3-4	3-6	4-5	5-6	6-7
τ nueva	1.500	1.071	2.357	2.357	1.357	2.929	1.071	2.357	1.500	2.929

En la segunda iteración, desde 6 (tras $1-7$) las probabilidades pasan a $p_{62} = 0.434$, $p_{63} = 0.086$, $p_{65} = 0.480$: el aprendizaje favorece la arista correcta, pero lentamente, porque la hormiga A también reforzó la trampa $\{2, 6\}$. Si se usa un esquema elitista como MMAS, en el que solo deposita la mejor ruta (C), entonces $\tau_{62} = 0.5$, $\tau_{63} = 0.5$ y $\tau_{65} = 1.5$, y las probabilidades quedan en 0.235, 0.059 y 0.706. Lección: **en grafos engañosos importa mucho quién deposita feromona**.

5. Pseudocódigo y ajustes principales

Algoritmo 1 ACO para el ciclo hamiltoniano (variante estilo MMAS)

Entrada: Grafo $G = (V, E)$ como listas de adyacencia; parámetros $m, \alpha, \beta, \rho, \tau_{\text{mín}}, \tau_{\text{máx}}$, límite de iteraciones T

Salida: Arreglo de n vértices que forma un ciclo hamiltoniano, o la mejor ruta parcial

$\tau_{ij} \leftarrow \tau_{\text{máx}}$ para toda $\{i, j\} \in E$
mejor $\leftarrow$ ruta vacía
for $t = 1$ **to** T **do**
    for $k = 1$ **to** m **do**
        elegir vértice inicial s al azar; $R_k \leftarrow [s]$; *visitados* $\leftarrow \{s\}$
        while $|R_k| < n$ **do**
            $i \leftarrow$ último vértice de R_k
            $N \leftarrow$ vecinos de i no visitados
            if $N = \emptyset$ **then**
                intentar reparar con rotación de Pósa; **si** no se puede, **break**
            else
                elegir $j \in N$ con probabilidad $\propto \tau_{ij}^{\alpha} \eta_{ij}^{\beta}$
                añadir j a R_k y a *visitados*
            end if
        end while
        $q_k \leftarrow$ (aristas válidas de R_k, incluido el cierre si existe)$/n$
        if $q_k = 1$ **then return** R_k ▷ ciclo hamiltoniano encontrado
        end if
        if $q_k >$ calidad de *mejor* **then** *mejor* $\leftarrow R_k$
        end if
    end for
    $\tau_{ij} \leftarrow (1 - \rho)\tau_{ij}$ para toda arista ▷ evaporación
    $\tau_{ij} \leftarrow \tau_{ij} + q_{mejor}$ para cada arista de *mejor* ▷ depósito
    acotar τ_{ij} al intervalo $[\tau_{\text{mín}}, \tau_{\text{máx}}]$
end for
return *mejor*

Tabla 2: Ajustes principales y valores de partida. Los valores de referencia son los que Dorigo y Stützle [3] recomiendan *para el TSP*; hay que calibrarlos para el FHCP.

Parámetro	Efecto	Referencia (TSP)
m (hormigas)	Más hormigas exploran más por iteración, pero cuestan más tiempo.	$m = n$ (AS, MMAS); $m = 10$ (ACS)
α	Peso de la feromona (lo aprendido). Muy alto: convergencia prematura.	1
β	Peso de la heurística (la estructura del grafo).	2 a 5
ρ	Evaporación. Alta: olvida rápido y explora; baja: explota lo aprendido.	0.5 (AS), 0.1 (ACS), 0.02 (MMAS)
$\tau_{\text{mín}}, \tau_{\text{máx}}$	Evitan que una arista quede con probabilidad casi 0 o casi 1.	MMAS
q_0	(ACS) probabilidad de elegir directamente el mejor vecino.	0.9
T	Límite de iteraciones o de tiempo; necesario para la comparación.	—

6. Aplicación al ciclo hamiltoniano: opciones, dificultades y dudas

6.1. Opciones de diseño

1. **[Propuesta] Grafo de construcción = grafo del problema**: las hormigas solo se mueven por aristas reales, así que nunca producen rutas ilegales; las fallas se reducen a «no llegó a todos» o «no cerró».

2. **[Propuesta] Calidad** = fracción de aristas válidas (Sección 3.5). Es equivalente a $1 - f(\pi)/n$ con la función de penalización de la Sección 2.4, lo que permite comparar ACO con GA, SA, DE y PSO con la misma métrica.

3. **[Propuesta] Variante MMAS o ACS**: el ejemplo muestra que el depósito de todas las hormigas refuerza trampas; un esquema elitista con límites de feromona es más apropiado.

4. **[Propuesta] Preprocesamiento**: fijar las aristas obligatorias de los vértices de grado 2 antes de lanzar la colonia.

5. **[Propuesta] Búsqueda local**: combinar ACO con rotaciones de Pósa o 2-opt restringido a aristas existentes, como se hace con ACO+búsqueda local en el TSP [3].

6.2. Dificultades previstas

- Las instancias del FHCP están construidas para engañar a heurísticas estándar [8]; muchas aristas «atractivas» no pertenecen a ningún ciclo.
- Si el número de hormigas es $m = n$ y el grafo tiene miles de vértices, una iteración cuesta del orden de n^2 pasos; para $n \approx 9500$ son unos 9×10^7 pasos.
- La feromona mide aristas, pero el problema es global: una arista puede estar en rutas casi completas y aun así no pertenecer a ningún ciclo hamiltoniano.
- Las rutas que fallan muy pronto dan poca información; la retroalimentación es escasa.

- Como referencia de dificultad, el ganador del concurso FHCP resolvió 985 de las 1001 instancias [9]; no se espera que una metaheurística simple las resuelva todas.

6.3. Dudas pendientes

1. **[Duda]** ¿El profesor espera trabajar con las 1001 instancias o con un subconjunto (por ejemplo, las más pequeñas)?
2. **[Duda]** ¿Qué límite de tiempo o de iteraciones usaremos para que la comparación sea justa entre los seis algoritmos?
3. **[Duda]** ¿Se permite combinar ACO con búsqueda local o reparación, o se pide la versión «pura» del algoritmo?
4. **[Duda]** ¿Cómo aplicar precisión, *recall* y F1 (tema del integrante 4) a un problema donde todas las instancias son hamiltonianas? Una propuesta es tratar «encontró el ciclo» como predicción positiva y comparar contra la solución oficial.
5. **[Duda]** ¿Acordamos en el equipo una única función de costo (aristas faltantes) para todos los algoritmos?

7. Fuentes consultadas

[1] M. Dorigo, V. Maniezzo y A. Colorni. «Ant System: Optimization by a colony of cooperating agents». *IEEE Trans. Systems, Man, and Cybernetics, Part B*, 26(1):29–41, 1996. `https://doi.org/10.1109/3477.484436`
Respalda: regla de transición probabilística, evaporación y depósito del Ant System.

[2] M. Dorigo y L. M. Gambardella. «Ant Colony System: A cooperative learning approach to the traveling salesman problem». *IEEE Trans. Evolutionary Computation*, 1(1):53–66, 1997. `https://doi.org/10.1109/4235.585892`
Respalda: variante ACS, actualización local y parámetro q_0.

[3] M. Dorigo y T. Stützle. *Ant Colony Optimization*. MIT Press, 2004. `https://mitpress.mit.edu/9780262042192/ant-colony-optimization/`
Respalda: visión general de ACO, estigmergia, MMAS, valores de parámetros recomendados para el TSP y combinación con búsqueda local.

[4] P. Larrañaga, C. M. H. Kuijpers, R. H. Murga, I. Inza y S. Dizdarevic. «Genetic algorithms for the travelling salesman problem: A review of representations and operators». *Artificial Intelligence Review*, 13(2):129–170, 1999. `https://doi.org/10.1023/A:1006529012972`
Respalda: representaciones de rutas (camino, adyacencia, ordinal) y su efecto en la búsqueda.

[5] C. A. Coello Coello. «Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey of the state of the art». *Computer Methods in Applied Mechanics and Engineering*, 191(11–12):1245–1287, 2002. `https://doi.org/10.1016/S0045-7825(01)00323-1`
Respalda: clasificación de técnicas para manejar restricciones (penalización, reparación, operadores que preservan factibilidad).

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 3.ª ed., MIT Press, 2009, sección 22.1. ISBN 978-0-262-03384-8.
Respalda: definiciones de grafo y representaciones por matriz y por listas de adyacencia, con sus costos.

[7] R. M. Karp. «Reducibility among combinatorial problems». En *Complexity of Computer Computations*, pp. 85–103, Plenum Press, 1972. https://doi.org/10.1007/978-1-4684-2001-2_9
Respalda: el ciclo hamiltoniano es NP-completo.

[8] M. Haythorpe. «FHCP Challenge Set: The first set of structurally difficult instances of the Hamiltonian cycle problem». *Bulletin of the ICA*, 83:98–107, 2018. https://arxiv.org/abs/1902.10352
Respalda: 1001 instancias hamiltonianas cuya dificultad proviene de la estructura, no solo del tamaño.

[9] Flinders Hamiltonian Cycle Project. «FHCP Challenge Set» (sitio oficial). https://sites.flinders.edu.au/flinders-hamiltonian-cycle-project/fhcp-challenge-set/
Respalda: rango de 66 a 9528 vértices, formato HCP y TOUR, resultado del concurso (985 de 1001 grafos resueltos por el equipo ganador).

[10] L. Pósa. «Hamiltonian circuits in random graphs». *Discrete Mathematics*, 14(4):359–364, 1976. https://doi.org/10.1016/0012-365X(76)90068-6
Respalda: técnica de rotación para extender caminos hacia ciclos hamiltonianos.

[11] J. C. Bean. «Genetic algorithms and random keys for sequencing and optimization». *ORSA Journal on Computing*, 6(2):154–160, 1994. https://doi.org/10.1287/ijoc.6.2.154
Respalda: representación por llaves aleatorias.

[12] I. A. Wagner y A. M. Bruckstein. «Hamiltonian(t) – An ant-inspired heuristic for recognizing Hamiltonian graphs». *Proc. Congress on Evolutionary Computation (CEC 99)*, 1999. https://doi.org/10.1109/CEC.1999.782656
Respalda: antecedente de hormigas aplicadas específicamente al ciclo hamiltoniano.

[13] T. Stützle y H. H. Hoos. «MAX–MIN Ant System». *Future Generation Computer Systems*, 16(8):889–914, 2000. https://doi.org/10.1016/S0167-739X(00)00043-1
Respalda: depósito solo de la mejor hormiga y límites de feromona.